

---

# From Bayes to BERT: A Comprehensive Benchmark for State-of-the-Art Intent Detection

---

**Tai Vu**  
taivu  
Stanford University  
taivu@stanford.edu

**Robert Yang**  
bobyang9  
Stanford University  
bobyang9@cs.stanford.edu

## Abstract

Intent detection is a crucial and challenging task in natural language understanding that powers a wide array of applications, from customer service chatbots to task-oriented dialog systems. This paper presents a comprehensive evaluation of various machine learning and deep learning models for automated intent classification. We implement and systematically compare traditional algorithms, including Naive Bayes, Softmax Regression, and Support Vector Machines, against more advanced deep learning architectures like Bidirectional Long Short-Term Memory (Bi-LSTM) and Bidirectional Encoder Representations from Transformers (BERT). Our experiments, conducted on the widely-used ATIS and SNIPS benchmark datasets, demonstrate the superior performance of a fine-tuned BERT model. This model achieves state-of-the-art results, attaining accuracy and F1 scores of over 99+% on both datasets. Furthermore, our analysis reveals that simpler, traditional models can yield surprisingly competitive results, highlighting important considerations for real-world deployment where computational complexity is a factor. This work not only establishes the effectiveness of transformer-based architectures for intent detection but also provides a valuable comparative analysis across a spectrum of model complexities.

## 1 Introduction

The ability to accurately discern user intent from natural language is a cornerstone of modern artificial intelligence, underpinning the functionality of technologies ranging from voice assistants and customer service chatbots to sophisticated task-oriented dialog systems. As the primary interface between humans and machines, the performance of these systems is directly tied to their capacity for high-fidelity Natural Language Understanding (NLU). A critical and often challenging component of NLU is intent detection: the task of correctly classifying a user’s goal or desired action from a textual command or query. While seemingly straightforward, this task is fraught with the nuances of human language, including semantic ambiguity, syntactic variation, and domain-specific jargon, making the pursuit of human-level accuracy a significant research challenge.

In recent years, the field has seen a rapid evolution of modeling techniques, progressing from traditional statistical methods to complex deep neural networks [1, 7]. While advanced architectures like Transformers have set new benchmarks for performance, their computational expense and complexity are not always justifiable in production environments. This creates a critical need for a systematic and comprehensive evaluation that not only identifies the highest-performing models but also clarifies the practical trade-offs across the entire spectrum of available techniques.

This paper addresses this need by conducting a rigorous comparative analysis of a wide array of models for automated intent detection. We systematically implement, train, and evaluate classic machine learning algorithms, including Naive Bayes, Softmax Regression, and Support Vector Machines, alongside powerful deep learning architectures, namely Bidirectional Long Short-Term Memory

(Bi-LSTM) and the state-of-the-art Bidirectional Encoder Representations from Transformers (BERT). By testing these models on two standard industry benchmarks, the ATIS and SNIPS datasets, we provide a definitive performance benchmark. Our findings confirm that a fine-tuned BERT model achieves near-perfect accuracy, exceeding 99%, while also revealing the surprisingly competitive performance of simpler, more traditional models. This work contributes a clear and comprehensive blueprint for building and selecting intent detection systems, offering valuable insights for both researchers and practitioners navigating the trade-offs between model complexity, computational cost, and classification accuracy.

## 2 Related Work

The field of natural language processing has been fundamentally transformed by recent breakthroughs in deep learning. Sophisticated neural architectures such as Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks [18, 20], and the revolutionary Transformer architecture [17, 23] have enabled unprecedented capabilities in understanding human language.

Within the subfield of intent detection, these advances have spurred significant progress. Early deep learning approaches successfully applied RNNs to design intent detection systems [9, 10, 11]. Subsequent research leveraged more complex models, including specialized LSTM and BERT architectures, to achieve progressively higher levels of accuracy [12, 13, 15, 14]. The advantage of these neural networks lies in their ability to learn deep vector encodings that capture the rich semantic meaning of user commands, leading to more accurate classification.

However, the state-of-the-art performance of these models often comes at the cost of significant computational complexity, which can be a barrier to their deployment in real-time dialog systems. This trade-off motivates a broader investigation into the performance spectrum of different models. While much research has focused on pushing the accuracy ceiling with complex neural networks, this study aims to provide a comprehensive comparison. We implement and evaluate powerful LSTM and BERT models while also benchmarking them against classic, computationally efficient machine learning algorithms, including Naive Bayes, Softmax Regression, and Support Vector Machines, to provide a clearer picture of the performance-versus-complexity landscape.

## 3 Dataset and Features

We used the following two popular datasets for intent classification benchmark.

- **Kaggle ATIS (Air Travel Information System)** [2] is a simplified version of the original MS-ATIS dataset, and contains English air travel related queries such as `airfare`, `flight`, and `ground service`. There are 4977 training examples and 893 test examples, which belong to 8 intents.
- **SNIPS** [3] consists of English day-to-day user commands like `get weather`, `play music`, and `book restaurant`. The dataset has 13802 training samples and 699 test samples. There are 7 user intents, and each contains 2400 examples.

For data preprocessing, we wrote a Python script to convert raw data files into pairs of inputs and outputs. Then, we removed punctuation, turn the sentences into lowercase, and split into lists of words by spaces, etc. Since we did not have validation data for ATIS and SNIPS, we wrote a program to split each provided training dataset into 95% for training and 5% for validation.

Regarding feature extraction, we implemented a Bag of Words approach. Specifically, we created a dictionary of all the words which appear in at least 5 training examples. Subsequently, for each input  $x$ , we generated a feature vector  $\phi(x)$ , where each entry  $x_j$  in  $\phi(x)$  is either the number of occurrences of the  $j$ -th word in  $x$  or a binary value indicating whether  $x$  contains the  $j$ -th word. This gave us two different sentence encodings that were appropriate for different machine learning models.

In addition, we implemented two neural word embeddings, including GloVe [6] and BERT [8]. These two encoders were pre-trained on large English corpora and are able to generate vector representations of input sentences in our datasets.

## 4 Methods

### 4.1 Machine Learning Algorithms

We developed several machine learning algorithms, including Naive Bayes, Softmax Regression, and Support Vector Machines.

**Naive Bayes** is a generative learning algorithm that models  $p(x | y)$  and  $p(y)$ , and then derive  $p(y | x)$  using Bayes rule. It makes an assumption that every feature  $x_j$ 's are conditionally independent given  $y$ , which means  $p(x_1, \dots, x_d | y) = \prod_{j=1}^d p(x_j | y)$ .

In the Naive Bayes algorithm with a Bernoulli event model, we have  $x_j = 1$  if the example  $x$  contain the  $j$ -th word of the dictionary; otherwise,  $x_j = 0$ . The parameters are  $\phi_{i|y=k} = p(x_j = 1 | y = k)$  and  $\phi_k = p(y = k)$ . Maximum likelihood estimates with Laplace smoothing give us  $\phi_{i|y=k} = \frac{1 + \sum_{i=1}^n 1\{x_j^{(i)} = 1 \wedge y^{(i)} = k\}}{2 + \sum_{i=1}^n 1\{y^{(i)} = k\}}$  and  $\phi_k = \frac{\sum_{i=1}^n 1\{y^{(i)} = k\}}{n}$ .

$$\begin{aligned} p(x_j = 1 | y = k) &= \frac{\sum_{i=1}^n 1\{x_j^{(i)} = 1 \wedge y^{(i)} = k\}}{\sum_{i=1}^n 1\{y^{(i)} = k\}} \\ p(y = k) &= \frac{\sum_{i=1}^n 1\{y^{(i)} = k\}}{n} \end{aligned}$$

In the Naive Bayes algorithm with a Multinomial event model, we let  $V$  denote the dictionary,  $x_j \in \{1, 2, \dots, |V|\}$  denote the identity of the  $j$ -th word in the input, and  $d_i$  denote the number of words in the  $i$ -th training example. The parameters are  $\phi_{l|y=k} = p(x_j = l | y = k)$  and  $\phi_k = p(y = k)$ . Maximum likelihood estimates with Laplace smoothing give us  $\phi_{l|y=k} = \frac{1 + \sum_{i=1}^n \sum_{j=1}^{d_i} 1\{x_j^{(i)} = l \wedge y^{(i)} = k\}}{|V| + \sum_{i=1}^n 1\{y^{(i)} = k\} d_i}$  and  $\phi_k = \frac{\sum_{i=1}^n 1\{y^{(i)} = k\}}{n}$ .

$$\begin{aligned} p(x_j = l | y = k) &= \frac{1 + \sum_{i=1}^n \sum_{j=1}^{d_i} 1\{x_j^{(i)} = l \wedge y^{(i)} = k\}}{|V| + \sum_{i=1}^n 1\{y^{(i)} = k\} d_i} \\ p(y = k) &= \frac{\sum_{i=1}^n 1\{y^{(i)} = k\}}{n} \end{aligned}$$

**Softmax Regression** is a generalization of the Logistic Regression algorithm for multi-class datasets. It uses the softmax function  $\frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$  for  $i = 1, \dots, k$ . Let  $n$  be the number of training examples,  $d$  be the number of the features, and  $k$  be the number of classes. The algorithm estimates  $p(y | x; \theta) = h_\theta(x)$  with the  $i$ -th element being  $\frac{\exp(W_i^T x + b_i)}{\sum_{j=1}^k \exp(W_j^T x + b_j)}$ , where  $W \in \mathbb{R}^{k \times d}$  and  $b \in \mathbb{R}^k$  are the weights, and  $x \in \mathbb{R}^d$  is the input. During training, the weights are updated through the equation  $W = W - \alpha(\hat{y} - y)X$ , where  $\alpha \in \mathbb{R}$  is a learning rate,  $\hat{y} \in \mathbb{R}^{k \times n}$  is the predicted probabilities,  $y \in \mathbb{R}^{k \times n}$  is the one-hot encoding of the labels, and  $X \in \mathbb{R}^{n \times d}$  is the input matrix. The biases are updated through the equation  $b = b - \alpha \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})$ .

**Support Vector Machine (SVM)** [26] is an algorithm that fits the data through maximizing the margin of separation between different classes. It conducts inference through the following formula:  $h_{w,b}(x) = g(w^T x + b)$ , where  $w$  are the weights,  $b$  are the biases, and  $g(z)$  is a function that outputs 1 if  $z \geq 0$  and outputs -1 otherwise. It conducts optimization through the following objective:  $\min_{\gamma, w, b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$  so that  $y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i$  for  $i = 1, \dots, n$  and that  $\xi_i \geq 0$  for  $i = 1, \dots, n$ .

For this project, we used a multi-class variant of SVM using the one-vs-one scheme, where a model is created to distinguish every possible pair of two classes.

## 4.2 Deep Learning Models

In addition to the above traditional machine learning techniques, we developed two neural networks for intent classification.

**Long short-term memory (LSTM)** is a model frequently used to process sequences of data. It is based on the recurrent neural network (RNN), where the hidden state at the previous timestep, along with the new input at the current timestep, can influence the current hidden state. LSTM has a cell memory with three gates (forget gate, input gate, and output gate), to preserve information over long sequences. We used Bidirectional LSTM (Bi-LSTM), which involved two LSTM models trained in opposite directions so that more context is provided.

**Bidirectional Encoder Representations from Transformers (BERT)** [8] is a state-of-the-art model for natural language processing. Its encoder acts as a feature extractor that provides vector embeddings for input sequences. The encoder leverages the Transformer architecture [17] to learn bidirectional representations from input texts by jointly conditioning on both left and right contexts in all layers. We took a BERT model pre-trained on a large corpus and fine-tuned its weights on our intent detection datasets.

Link to our project: <https://github.com/bobyang9/NLU-for-Intent-Classification>

## 5 Experiments and Results

### 5.1 Evaluation Metrics

Since we are working with a classification problem, we measure the performance of our algorithms using classification accuracy and F1 scores. The formula for classification accuracy is  $\frac{TP+TN}{TP+TN+FP+FN}$ , and the formula for F1 score is  $2 * \frac{precision * recall}{precision + recall}$ , also known as  $\frac{TP}{TP + \frac{1}{2}(FP+FN)}$ , where TP = true positive, TN = true negative, FP = false positive, and FN = false negative. We use classification accuracy to see how well our model is performing overall and we use F1 score to see if we are correctly balancing precision and recall.

### 5.2 Performance of Machine Learning Algorithms on Validation Data

We implemented Naive Bayes and Softmax Regression from scratch, and used the scikit-learn implementation for SVM [16]. The Softmax Regression was run for 200 iterations with a learning rate of 0.001. We used the RBF kernel for SVM. We trained the models on ATIS and SNIPS with different feature extractors and then recorded their performances on validation sets in Tables 1 and 2.

The three algorithms had high performances on the SNIPS dataset. Softmax Regression with Bag of Words achieved 98.46% validation accuracy and 0.9869 validation F1 score. SVM with BERT encodings also performed well, achieving 98.12% validation accuracy and 0.9812 F1 score. Most of the other combinations achieved validation accuracies and F1 scores in the range of 96.5 – 98%. The same trends applies for ATIS. Softmax Regression with BERT embeddings had 96.69% validation accuracy and 0.9686 validation F1 score, while SVM with BERT embeddings had 96.28% accuracy and 0.9568 F1 score.

### 5.3 Performance of Deep Learning Models on Validation Data

We implemented the Bi-LSTM using TensorFlow [22] and Keras [27] using the architecture in Figure 1. An additional BatchNorm [25] layer was added for the model trained ATIS because ATIS was less stable (The BatchNorm layer was not included in the model for SNIPS as it hurt performance). After noticing that the model was overfitting, we added a Dropout [24] layer with 50% probability to curb the overfitting. We trained the LSTM until early stopping defined by the training loss not improving for three epochs. We used the Adam optimization algorithm with a learning rate of 0.0001 and a batch size of 64 (a batch size that fits into memory).

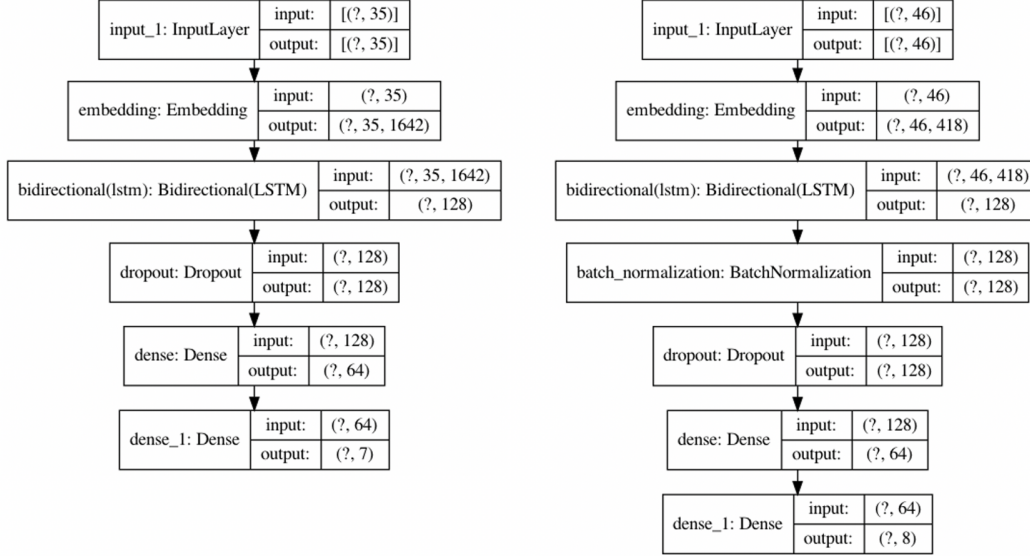
For the BERT implementation, we developed a BERT neural network using PyTorch [19] and Hugging Face [21], and then fine-tuned its pre-trained weights on our datasets. The model’s encoder receives input sentences and outputs a 768 dimensional encoding vector. For this classification task,

| Algorithm                 | Features     | Val Acc       | Val. F1       |
|---------------------------|--------------|---------------|---------------|
| Naive Bayes (Bernoulli)   | Bag of Words | 0.9783        | 0.9782        |
| Naive Bayes (Multinomial) | Bag of Words | 0.9739        | 0.9737        |
| Softmax Regression        | Bag of Words | <b>0.9846</b> | <b>0.9869</b> |
| Softmax Regression        | GloVe        | 0.9725        | 0.9721        |
| Softmax Regression        | BERT         | 0.9667        | 0.9663        |
| Support Vector Machines   | Bag of Words | 0.9290        | 0.9291        |
| Support Vector Machines   | GloVe        | 0.9797        | 0.9797        |
| Support Vector Machines   | BERT         | <b>0.9812</b> | <b>0.9812</b> |
| LSTM                      | One-hot      | <b>0.9812</b> | <b>0.9811</b> |
| LSTM                      | GloVe        | 0.9609        | 0.9610        |
| LSTM                      | BERT         | 0.9696        | 0.9696        |
| BERT                      | N/A          | <b>0.9913</b> | <b>0.9913</b> |
| Naive Bayes (Bernoulli)   | Bag of Words | 0.9256        | 0.9247        |
| Naive Bayes (Multinomial) | Bag of Words | 0.9256        | 0.9236        |
| Softmax Regression        | Bag of Words | 0.9587        | 0.9552        |
| Softmax Regression        | GloVe        | 0.9132        | 0.8940        |
| Softmax Regression        | BERT         | <b>0.9669</b> | <b>0.9686</b> |
| Support Vector Machines   | Bag of Words | 0.9215        | 0.9079        |
| Support Vector Machines   | GloVe        | 0.9463        | 0.9397        |
| Support Vector Machines   | BERT         | <b>0.9628</b> | <b>0.9568</b> |
| LSTM                      | One-hot      | <b>0.9711</b> | <b>0.9692</b> |
| LSTM                      | GloVe        | 0.9132        | 0.8890        |
| LSTM                      | BERT         | 0.8967        | 0.8960        |
| BERT                      | N/A          | <b>0.9959</b> | <b>0.9962</b> |

**Table 1:** Performance comparison of different models  
(upper: SNIPS, lower: ATIS)

| Algorithm                 | Feature Extraction | Validation Accuracy | Validation F1 score |
|---------------------------|--------------------|---------------------|---------------------|
| Naive Bayes (Bernoulli)   | Bag of Words       | 0.9256              | 0.9247              |
| Naive Bayes (Multinomial) | Bag of Words       | 0.9256              | 0.9236              |
| Softmax Regression        | Bag of Words       | 0.9587              | 0.9552              |
| Softmax Regression        | GloVe              | 0.9132              | 0.8940              |
| Softmax Regression        | BERT               | <b>0.9669</b>       | <b>0.9686</b>       |
| Support Vector Machines   | Bag of Words       | 0.9215              | 0.9079              |
| Support Vector Machines   | GloVe              | 0.9463              | 0.9397              |
| Support Vector Machines   | BERT               | <b>0.9628</b>       | <b>0.9568</b>       |
| LSTM                      | One-hot            | <b>0.9711</b>       | <b>0.9692</b>       |
| LSTM                      | GloVe              | 0.9132              | 0.8890              |
| LSTM                      | BERT               | 0.8967              | 0.8960              |
| BERT                      | N/A                | <b>0.9959</b>       | <b>0.9962</b>       |

**Table 2:** Performance comparison of machine learning models on ATIS.



**Figure 1:** LSTM Model Architecture for SNIPS (left) and ATIS (right)

| Algorithm | Feature Extraction | Validation Accuracy | Validation F1 score |
|-----------|--------------------|---------------------|---------------------|
| LSTM      | One-hot            | <b>0.9812</b>       | <b>0.9811</b>       |
| LSTM      | GloVe              | 0.9609              | 0.9610              |
| LSTM      | BERT               | 0.9696              | 0.9696              |
| BERT      | N/A                | <b>0.9913</b>       | <b>0.9913</b>       |

**Table 3:** Performance comparison of deep learning models on SNIPS.

we added a fully connected layer and a softmax layer following the encoder. BERT is a bigger model, so we used a smaller batch size of 16. The optimizer and the learning rate remain the same.

We trained Bi-LSTM and BERT models on ATIS and SNIPS and recorded their validation performances in Tables 3 and 4. Both models had high performances on SNIPS. BERT achieved 99.13% accuracy and 0.9913 F1 score, while LSTM with one-hot word encodings achieved 98.12% accuracy and 0.9811 F1 score. Similar trends were observed for ATIS. BERT achieved 99.59% accuracy and 0.9962 F1 score, while LSTM with one-hot word encodings achieved 97.11% accuracy and 0.9692 F1 score.

#### 5.4 Analysis

We found that BERT was the highest performer among the models that we tried. The reason might be that its Transformer architecture had the attention mechanism that allows it to identify the important parts of the text (even those very far away) which guided it to make more informed classifications. From the confusion matrix in Figure 2, we can see that BERT had a good performance across all labels and did not get confused by any pairs of classes.

| Algorithm | Feature Extraction | Validation Accuracy | Validation F1 score |
|-----------|--------------------|---------------------|---------------------|
| LSTM      | One-hot            | <b>0.9711</b>       | <b>0.9692</b>       |
| LSTM      | GloVe              | 0.9132              | 0.8890              |
| LSTM      | BERT               | 0.8967              | 0.8960              |
| BERT      | N/A                | <b>0.9959</b>       | <b>0.9962</b>       |

**Table 4:** Performance comparison of deep learning models on ATIS.

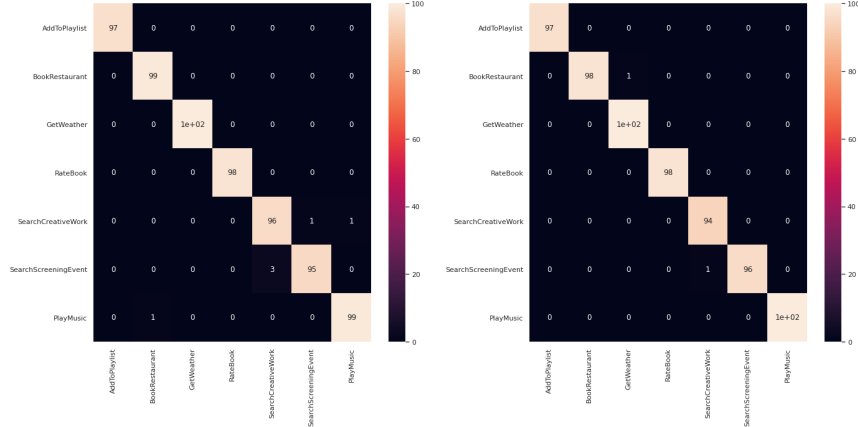


Figure 2: Confusion Matrix for BERT (left) and LSTM (right).

| Test Dataset | Test Accuracy | Test F1 score |
|--------------|---------------|---------------|
| SNIPS        | 0.9900        | 0.9900        |
| ATIS         | 0.9938        | 0.9951        |

Table 5: Performance on test data.

However, we found that Softmax Regression with Bag of Words feature extraction, a much simpler model, actually came pretty close in performance. In fact, it performed better than Bi-LSTM, which is considered a complex deep learning model. Another notable point is that other algorithms like Naive Bayes and SVMs also had great performance despite their simplicity.

One example that BERT predicted incorrectly was *"i'm going to leave philadelphia and i want to go to san francisco and i want to fly first class american and i want a stop in dallas can you please tell me what type of aircraft you will be flying"*. Its ground truth label was *atis\_aircraft*, but the system predicted *atis\_flight*. The algorithm was perhaps misled by the preponderance of destination terms like *"philadelphia"*, *"san francisco"*, and *"dallas"*, as well as many phrases related to booking a flight, including *"i want a stop in ..."*, *"i want to fly first class ..."*, etc. Only the last segment (*"can you please tell me what type of aircraft you will be flying"*) had to do with aircraft, so the algorithm may have believed that the category was more likely to be *atis\_flight*.

Another incorrect prediction was *"what is the schedule of ground transportation from the airport in philadelphia into downtown"*. The program outputted *atis\_flight\_time* instead of the correct label *atis\_ground\_service*. The algorithm may have put too much weight on *"schedule"*, so it believed that the text was about flight time, while the actual emphasis was on *"ground transportation"*.

Another interesting pattern is that fine-tuning the BERT model leads to a better outcome than combining a fixed pre-trained BERT or GloVe encoder with a classifier. The reason might be that the distribution of words in the pre-trained English corpora and in our intent datasets are different, because the ATIS and SNIPS examples only belong to limited domains. Therefore, the fine-tuning process updates the weights in BERT encoder to adapt better to our training sets.

## 5.5 Performance on Test Data

Finally, we evaluated our best model, the fine-tuned BERT, on SNIPS and ATIS test sets and recorded the performance in table 5. The model performed very well, achieving 99.00% test accuracy and 0.9900 test F1 score on SNIPS and 99.38% test accuracy and 0.9951 test F1 score on ATIS.

## 6 Conclusion

In this study, we conducted a comprehensive evaluation of various machine learning and deep learning algorithms for the critical task of intent detection. Our empirical results demonstrate that a fine-tuned

BERT model achieves state-of-the-art performance on both the ATIS and SNIPS benchmark datasets, attaining exceptional accuracy scores of over 99% and F1 scores exceeding 0.99. This outcome underscores the power of large, pre-trained transformer models to understand the nuances of user commands with near-human accuracy.

Furthermore, our analysis provides a crucial insight: simpler, traditional models such as Softmax Regression with Bag of Words features can yield surprisingly competitive results, in some cases outperforming more complex neural architectures like LSTMs. This finding is significant for real-world applications, highlighting a practical trade-off between state-of-the-art performance and computational efficiency. The study confirms that while BERT represents the pinnacle of accuracy, computationally lighter models remain highly viable alternatives for systems where deployment constraints are a primary concern.

## 7 Future Work

Building on the insights from this research, we have identified several promising directions for future work:

- **Hyperparameter Optimization for Efficient Models:** We plan to perform extensive hyperparameter tuning on the most promising traditional models, such as Softmax Regression and SVM. The goal is to systematically optimize these computationally efficient models and determine the maximum performance they can achieve, potentially narrowing the gap with deep learning approaches.
- **Generalization to More Complex Datasets:** To further validate the robustness of our models, we intend to evaluate them on more challenging benchmarks like CLINC150 [4]. This dataset is significantly larger and is designed to test a model's ability to handle a wider range of intents and, crucially, to identify out-of-scope queries, which is a critical capability for production-ready dialog systems.
- **Expansion to Joint Intent and Slot Filling:** A natural and significant extension of this work is to move from pure intent detection to a joint intent classification and slot filling architecture. This integrated task more closely mirrors the complete NLU challenge in task-oriented dialog systems. The ultimate goal is to deploy these advanced, multi-task models into a functional chatbot system to evaluate their performance in a live, interactive environment.

## References

- [1] Zhang, Z., Takanobu, R., Zhu, Q., Huang, M., & Zhu, X. (2020). Recent advances and challenges in task-oriented dialog systems. *Science China Technological Sciences*, 1-17.
- [2] Hemphill, C. T., Godfrey, J. J., & Doddington, G. R. (1990). The ATIS spoken language systems pilot corpus. In *Speech and Natural Language: Proceedings of a Workshop Held at Hidden Valley, Pennsylvania, June 24-27, 1990*.
- [3] Coucke, A., Saade, A., Ball, A., Bluche, T., Caulier, A., Leroy, D., ... & Primet, M. (2018). Snips voice platform: an embedded spoken language understanding system for private-by-design voice interfaces. *arXiv preprint arXiv:1805.10190*.
- [4] Larson, S., Mahendran, A., Peper, J. J., Clarke, C., Lee, A., Hill, P., ... & Mars, J. (2019). An evaluation dataset for intent classification and out-of-scope prediction. *arXiv preprint arXiv:1909.02027*.
- [5] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.
- [6] Pennington, J., Socher, R., & Manning, C. D. (2014, October). Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)* (pp. 1532-1543).
- [7] Vu, T., Wen, E., & Nehoran, R. (2020). How Not to Give a FLOP: Combining Regularization and Pruning for Efficient Inference. *ArXiv*, abs/2003.13593.
- [8] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.



- [9] Peng, B., & Yao, K. (2015). Recurrent neural networks with external memory for language understanding. arXiv preprint arXiv:1506.00195.
- [10] Mesnil, G., Dauphin, Y., Yao, K., Bengio, Y., Deng, L., Hakkani-Tur, D., ... & Zweig, G. (2014). Using recurrent neural networks for slot filling in spoken language understanding. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 23(3), 530-539.
- [11] Mesnil, G., Dauphin, Y., Yao, K., Bengio, Y., Deng, L., Hakkani-Tur, D., ... & Zweig, G. (2014). Using recurrent neural networks for slot filling in spoken language understanding. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 23(3), 530-539.
- [12] Niu, P., Chen, Z., & Song, M. (2019). A novel bi-directional interrelated model for joint intent detection and slot filling. arXiv preprint arXiv:1907.00390.
- [13] Wang, Y., Shen, Y., & Jin, H. (2018). A bi-model based rnn semantic frame parsing model for intent detection and slot filling. arXiv preprint arXiv:1812.10235.
- [14] Vu, T., & Yang, R. (2025). GANime: Generating Anime and Manga Character Drawings from Sketches with Deep Learning.
- [15] Qin, L., Che, W., Li, Y., Wen, H., & Liu, T. (2019). A stack-propagation framework with token-level intent detection for spoken language understanding. arXiv preprint arXiv:1909.02188.
- [16] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Vanderplas, J. (2011). Scikit-learn: Machine learning in Python. *the Journal of machine Learning research*, 12, 2825-2830.
- [17] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems* (pp. 5998-6008).
- [18] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.
- [19] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Desmaison, A. (2019). Pytorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems* (pp. 8026-8037).
- [20] Vu, T., & Yang, R. (2025). BERT-VQA: Visual Question Answering on Plots.
- [21] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Brew, J. (2019). HuggingFace's Transformers: State-of-the-art Natural Language Processing. ArXiv, arXiv-1910.
- [22] Abadi, M., Agarwal, A., Barham, B., Brevdo, E., Chen, Z., Citro, C., ... & Zheng, X. (2015). Tensorflow: Large-Scale Machine Learning on Heterogeneous Systems. Software available from tensorflow.org.
- [23] Sun, M., Vu, T., & Wang, A. (2025). Privacy Preserving Inference of Personalized Content for Out of Matrix Users.
- [24] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1), 1929-1958.
- [25] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. arXiv preprint arXiv:1502.03167.
- [26] Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine learning*, 20(3), 273-297.
- [27] Chollet, F., & others. (2015). Keras. GitHub. Retrieved from <https://github.com/fchollet/keras>